

Module Systems in Node.js

1. CommonJS (CJS)

- **Default** module system in Node.js (used before ES6).
- Used with `.js` files.

☒ Exporting

```
// math.js
const add = (a, b) => a + b;
module.exports = add;
```

☒ Importing

```
// app.js
const add = require('./math');
console.log(add(2, 3)); // 5
```

☒ Characteristics

- Synchronous (loaded at runtime).
- Works only in Node.js (not in browsers directly).
- `require`, `module.exports` are CommonJS keywords.

2. ES6 Modules (ESM)

- Modern standard (introduced in ES6).
- Used with `.mjs` files or set `"type": "module"` in `package.json`.

☒ Exporting

```
// math.mjs
export const add = (a, b) => a + b;
```

☒ Importing

```
// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

 Characteristics

- Asynchronous (loaded using promises).
- Works in both browser and Node.js.
- Strict mode by default.

 Key Differences

Feature	CommonJS	ES6 Modules
File extension	<code>.js</code>	<code>.mjs</code> or <code>.js</code> with <code>"type": "module"</code>
Syntax	<code>require</code> , <code>module.exports</code>	<code>import</code> , <code>export</code>
Loading	Synchronous	Asynchronous
Scope	Local to file	Strict mode, block-scoped
Use in browser	 Not directly	 Yes

 CommonJS Module System

◇ What is CommonJS?

- A module system used in Node.js.
- Helps organize code into files (modules).
- Encourages reusable and maintainable code.
- Used mostly in server-side JavaScript.

◇ Key Concepts

Concept	Description
<code>module.exports</code>	Exports a value from a module
<code>require()</code>	Imports a module into another file (return value is -> <code>module.exports</code>)

◇ Exporting from a Module

```
// math.js
function add(a, b) {
  return a + b;
}

module.exports = add;
```

◇ Importing a Module

```
// app.js
const add = require('./math'); // same as './math.js'
console.log(add(2, 3));
```

◇ Export Multiple Values

```
// utils.js
function greet(name) {
  return `Hello, ${name}`;
}

function bye(name) {
  return `Goodbye, ${name}`;
}

module.exports = { greet, bye };
```

```
// app.js
const utils = require('./utils');
console.log(utils.greet("Afnur"));
```

◇ Features

- ☒ Synchronous loading
- ☒ Works only in Node.js (not in browser by default)
- ☒ Module caching (loaded only once)

◇ Summary

- Use `require()` to import.
- Use `module.exports` to export.
- Used mainly in backend Node.js apps.
- Synchronous (blocks code until file is loaded)

`module.exports` vs `exports` in Node.js

What are they?

Both `module.exports` and `exports` are used to **export** code (functions, objects, variables) from a Node.js module so that it can be used in another file.

`module.exports`

- It is the actual object that gets **returned when a module is required**.
- You can assign any value to it (object, function, class, etc.).

```
// file: greet.js
function sayHello() {
  console.log("Hello!");
}

module.exports = sayHello;
```

```
// file: app.js
const greet = require('./greet');
greet(); // Hello!
```

exports

- It's a **shortcut** to `module.exports`.
- Works **only when you are adding properties** (not replacing the whole object).

```
// file: greet.js
exports.sayHello = function () {
  console.log("Hello!");
};
```

```
// file: app.js
const greet = require('./greet');
greet.sayHello(); // Hello!
```

! Important Rule

If you do this :

```
exports = function () {}; //  Doesn't work
```

You break the link with `module.exports`, and nothing is exported.

☒ Use this instead:

```
module.exports = function () {}; // ☒ Works
```

Best Practice

Use `module.exports` when:

- You export a **single thing** (function, class, object).

Use `exports` when:

- You export **multiple properties**.

Summary

Feature	<code>module.exports</code>	<code>exports</code>
Type	Object	Reference to <code>module.exports</code>
Can replace whole export	☒ Yes	☒ No
Add properties	☒ Yes	☒ Yes
Common use	One main export (e.g. function)	Multiple named exports

`module` Object in Node.js

What is `module`?

- In Node.js, every file is treated as a **separate module**.
- Node wraps each module in a function to give access to `module`, `exports`, `require`, etc.

Key Properties of `module`

Property	Description
<code>module.exports</code>	 Main object used to export values
<code>module.id</code>	 ID of the module (usually the file path)
<code>module.filename</code>	 Full path of the module file
<code>module.loaded</code>	☒ <code>true</code> if the module is loaded
<code>module.parent</code>	 Module that required this module
<code>module.children</code>	 Modules that were required by this module

Internal Wrapper Function

Every module is wrapped like:

```
(function(exports, require, module, __filename, __dirname) {  
  // Your code here  
});
```

Module Wrapper Function – Node.js

What is the Module Wrapper?

- In Node.js, every module (file) is **wrapped** in a function before execution.
- This gives each module **its own scope** (like a private sandbox).

Actual Wrapper Function

```
(function (exports, require, module, __filename, __dirname) {  
  // Your module code here  
});
```

- This is how Node.js internally wraps your code when loading a module.

Parameters Explained

Parameter	Meaning
<code>exports</code>	Shortcut to <code>module.exports</code> to export things
<code>require</code>	Used to import other modules
<code>module</code>	Represents the current module
<code>__filename</code>	Absolute path of the current file
<code>__dirname</code>	Directory path of the current file

Why Use a Wrapper?

- To **avoid polluting** the global scope.
- To provide module-level variables like `exports`, `require`, etc.

Example

```
console.log(arguments.callee.toString());
```

- ☒ This will show the wrapper function around your code.

ES6 Modules in Node.js

◇ What are ES6 Modules?

- A modern way to **import** and **export** JavaScript code.
- Uses **import** and **export** keywords instead of `require()` and `module.exports`.

☑ Enable ES6 Modules in Node.js:

To use ES6 modules:

1. Use **.mjs** extension

or

2. Add this line in `package.json`:

```
"type": "module"
```

🔄 Exporting in ES6:

1. **Named Export:**

```
export const name = "Afnur";  
export function greet() { ... }
```

2. **Default Export:**

```
export default function () { ... }
```

🔄 Importing in ES6:

1. **Named Import:**

```
import { name, greet } from './file.js';
```

2. **Default Import:**

```
import greet from './file.js';
```

3. **Renaming Import:**

```
import { greet as hello } from './file.js';
```

⚠ Important Points:

- **File paths must include .js** when importing.
- Top-level **await** is allowed in ES Modules (not in CommonJS).
- Cannot use **__dirname** or **__filename** directly – use:

```
import { fileURLToPath } from 'url';
import { dirname } from 'path';

const __filename = fileURLToPath(import.meta.url); // or
import.meta.filename (no need to import anything).
const __dirname = dirname(__filename); // or import.meta.dirname (no need to
import anything).
```

🔄 ES Modules vs CommonJS

Feature	ES Modules	CommonJS
Import	<code>import</code>	<code>require()</code>
Export	<code>export</code>	<code>module.exports</code>
File Extension	<code>.js</code> or <code>.mjs</code>	<code>.js</code>
Async/Top-level	Supported	Not supported
File Loading/Reading	Asynchronously	Synchronously
Imports Hoisting	Hoisted	Not hoisted
Strict mode	Enabled	Not enabled

☑ Use When:

- Writing modern JavaScript.
- Targeting browsers and modern Node.js apps.

🔗 Different Types of Modules in Node.js

Node.js has three main types of modules:

1. 📦 Core Modules

- Built into Node.js (no installation needed).
- Comes with Node.js by default.
- Examples:
 - `fs` – File system operations
 - `http` – Web server creation

- `path` – File path handling
- `os` – OS-related info
- `events` – Event handling

```
const fs = require('fs');
```

2. Local Modules

- Created by the developer for custom logic.
- Stored in local files.
- Helps in code reuse and modular design.

```
// myModule.js
exports.sayHello = () => {
  console.log("Hello!");
};

// index.js
const myModule = require('./myModule');
myModule.sayHello();
```

3. Third-Party Modules

- Installed using npm (Node Package Manager).
- Developed by others and shared via npm.
- Examples:
 - `express` – Web framework
 - `mongoose` – MongoDB object modeling
 - `lodash` – Utility functions

```
npm install express
```

```
const express = require('express');
```

Summary

Type	Source	Example
Core Module	Node.js built-in	<code>fs</code> , <code>http</code>
Local Module	Your own code	<code>./myModule.js</code>
Third-Party Module	Installed via npm	<code>express</code> , <code>lodash</code>

package.json vs package-lock.json in Node.js

package.json

◇ What is it?

The main configuration file for any Node.js project.

 Contains:

- **name** – Project name
- **version** – Project version
- **description** – Info about the project
- **scripts** – Commands to run (like start, test)
- **dependencies** – Libraries needed for the app to run (for production)
- **devDependencies** – Tools needed only during development (like testing tools)
- **main** – Entry file (like `index.js`)

 Example:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

package-lock.json

◇ What is it?

Auto-generated file created when `npm install` runs. It locks versions of installed packages.

 Purpose:

- Ensures **same versions** of dependencies are installed every time
- Improves **security** and **consistency**
- Used by `npm` to track **exact versions** of dependencies and sub-dependencies

Key Difference:

Feature	<code>package.json</code>	<code>package-lock.json</code>
Editable	☒ Yes (manually)	☒ No (auto-generated)
Purpose	Project info & dependencies	Exact version locking
Needed for install	☒ Yes	☒ Yes (for consistency)

☒ Summary

- Use `package.json` to **define your project**
- Let `package-lock.json` ensure **stable installs**

Shebang (#!)

◇ What is Shebang?

- A **special line** at the **very top** of a script file.
- Tells the **OS** which interpreter to use (e.g., Python, Bash).

◇ Syntax:

```
#!/<path-to-interpreter>
```

Example:

```
#!/usr/bin/python3
print("Hello, world!")
```

- This runs the script using Python 3.

◇ Why it's important?

- ☒ Makes the script **executable** directly (e.g., `./script.py`)
- ☒ Helps in **cross-platform** scripting
- ☒ Avoids needing to type `python script.py` every time

◇ Common Shebangs:

Language	Shebang
Bash	<code>#!/bin/bash</code>

Language	Shebang
Python	<code>#!/usr/bin/python3</code> or <code>#!/usr/bin/env python3</code>
Node.js	<code>#!/usr/bin/env node</code>

⚠ Notes:

- Shebang only works on **Unix/Linux**, not needed on Windows.
- Must be the **first line** of the file, no space before `#!`.

Library vs CLI Packages

Aspect	Library Package	CLI (Command-Line Interface) Package
 Usage	Imported in code (e.g., <code>require</code> , <code>import</code>)	Run from terminal/command line
 Example	<code>lodash</code> , <code>express</code> , <code>axios</code>	<code>nodemon</code> , <code>eslint</code> , <code>npm</code> , <code>create-react-app</code>
 Purpose	Helps build applications with reusable functions	Provides command-line tools to aid development
 Integration	Used inside app code	Used during development/deployment

Global vs Local Packages (Node.js)

Type	Installed with	Available in	Use case	Example
 Global	<code>npm install -g <package></code>	Anywhere on your system	CLI tools	<code>npm</code> , <code>nodemon</code>
 Local	<code>npm install <package></code>	Only in current project dir	Project-specific dependencies	<code>express</code> , <code>mongoose</code>

 Check Commands:

```
# Check global packages
npm list -g --depth=0

# Check local packages
npm list --depth=0
```

Summary

- **Library packages** are for coding logic.
- **CLI packages** are for command-line tasks.
- **Global packages** work system-wide.
- **Local packages** are specific to your project.



What is **npx**? and How npx Works in Node.js

☑ What is npx?

- **npx** is a Node.js tool used to **execute packages directly** without installing them globally.
- It comes bundled with **npm v5.2.0 and above**.

🔍 Step-by-Step Search & Execution Process

◇ Step 1: Local Project Check

- Searches for **package.json** in the current directory.
- Looks for the package name under the **name** key.
- Then looks for the executable file under the **bin** key.

◇ Step 2: Local Bin Folder

- Checks for the package (e.g., **hello**) inside:

```
./node_modules/.bin/
```

- If found, executes it directly.

◇ Step 3: Global npm Installation

- If not found locally, searches in the **globally installed packages**.
- If found, executes the global package.

◇ Step 4: npx Cache

- If not found globally, checks the **npx cache folder**.
- If found, prompts and runs it.
- If not found in cache, goes to Step 5.

◇ Step 5: npm Registry

- Searches for the package in the **npm registry**.
- Prompts: "Would you like to install?"
- If you agree:
 - Downloads the package.
 - Installs it temporarily.

- Runs the package and then removes it.

Use Case Example

```
npx create-react-app myApp
```

Executes the `create-react-app` without globally installing it.

Difference between `npm` and `npx`:

Feature	npm	npx
Purpose	Installs packages	Executes packages
Installation	Required (locally/globally)	Not required (can run directly)
Use Case	Managing dependencies	Running CLI tools or commands

Reading Files in Node.js

Node.js provides the `fs` module to interact with the file system.

1. Import the Module

```
const fs = require('fs'); // For callback-based methods
const fsPromises = require('fs/promises'); // For promise-based methods
```

OR

1. Importing Modules (ES6)

```
import fsPromises from 'fs/promises';
import fs from 'fs';
```

⚠ Note: Use only one module system to import. To use `import`, your file must be `.mjs` or `"type": "module"` must be in your `package.json`.

2. Read File – Promise Version (async/await)

```
try {
  const data = await fsPromises.readFile('file.txt', 'utf8'); //used ES6 import
  console.log(data);
} catch (err) {
```

```
    console.error(err);  
  }
```

- Use `try...catch` for error handling.

3. Read File – Callback Version

```
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) {  
    console.error(err);  
    return;  
  }  
  console.log(data);  
});
```

- `'utf8'` ensures readable string instead of raw buffer.
- `err` handles errors if file not found or permission denied.

4. Read File – Synchronous Version

```
const data = fs.readFileSync('file.txt', 'utf8'); //only in CommonJs  
console.log(data);
```

- Blocks the event loop ⚠ (not recommended for large files or servers).

Notes:

- `readFile` is **asynchronous** (non-blocking).
- `readFileSync` is **synchronous** (blocking).
- Prefer `fs/promises` for modern async/await usage.

Writing to a File in Node.js

1. Importing FS Module (ES6)

```
import { writeFile } from 'fs/promises';
```

2. Writing to a File (Async)

```
try {  
  await writeFile('output.txt', 'Hello, Node.js!', 'utf8');
```

```
    console.log('File written successfully!');
  } catch (err) {
    console.error('Error writing file:', err);
  }
```

Parameters:

- `'output.txt'`: Filename (creates if doesn't exist)
- `'Hello, Node.js!'`: Content to write
- `'utf8'`: Encoding (optional but recommended)

3. Overwriting vs Appending

- `writeFile()` ► **overwrites** the file by default
- To **append** content, use `appendFile()`:

```
import { appendFile } from 'fs/promises';

await appendFile('output.txt', '\nAppended text');
```

4. Error Handling

Always use try-catch or `.catch()` to handle file errors (e.g., permissions, missing path).

Example Folder Structure:

```
project/
├──
├── index.js
└── output.txt (created after running)
```

Other File & Directory Operations in Node.js

1. Create a File

```
import { writeFile } from 'fs/promises';

await writeFile('file.txt', 'Hello!');
```

2. Delete a File


```
import { unlink } from 'fs/promises';

await unlink('file.txt');
```

3. Create a Directory

```
import { mkdir } from 'fs/promises';

await mkdir('myFolder');
// Add { recursive: true } to create nested folders
```

4. Delete a Directory (Deprecated)

```
import { rmdir } from 'fs/promises';

await rmdir('myFolder');
// Deprecated: Use rm() with { recursive: true }
```

5. Delete Folder with Files

```
import { rm } from 'fs/promises';

await rm('myFolder', { recursive: true, force: true });
```

6. Read File Content

```
import { readFile } from 'fs/promises';

const data = await readFile('file.txt', 'utf8');
console.log(data);
```

7. Rename a File/Folder

```
import { rename } from 'fs/promises';

await rename('old.txt', 'new.txt');
```

8. Move (Rename) File or Folder

```
import { rename } from 'fs/promises';

await rename('old.txt', 'folder/new.txt');
```

- Renames or moves files or folders.
- The destination can be a different folder.

9. `copyFile()` — Copy a File

```
import { copyFile } from 'fs/promises';

await copyFile('source.txt', 'destination.txt');
```

10. `readdir()` - Reads a directory and returns a Array of files and folders

```
import { readdir } from 'fs/promises';

const files = await readdir('./my-folder'); // replace with your directory path
console.log(files); // Array of files and folders
```

11. `fs.watch()` — Watch File/Dir for Changes

```
import { watch } from 'fs';

watch('file.txt', (eventType, filename) => {
  console.log(`Event: ${eventType}, File: ${filename}`);
});
```

- Watches file for **changes, renames**, etc.
- Useful for auto-reloading or monitoring file changes.

12. `fs.stat()` — Get File Info

```
import { stat } from 'fs/promises';

const info = await stat('file.txt');
console.log(info.size); // File size in bytes
console.log(info.isFile()); // true if it's a file
```

- Provides metadata: size, timestamps, type (file or directory), etc.

